



Unit IV

OBJECT ORIENTED DESIGN PROCESS

Object Oriented Design Process:

Designing Business Layer:

Object Oriented Constraints Language (OCL):

Object Constraint Language (OCL) is a formal language used primarily in software engineering and software development, particularly in the context of modeling languages such as Unified Modeling Language (UML) and the Meta-Object Facility (MOF). OCL is part of the Object Management Group (OMG)'s family of specifications for model-driven development.

OCL is used to specify constraints and conditions on objects within a model. It provides a way to express rules and invariants that must be satisfied by the elements of a model. These constraints can include conditions on attribute values, relationships between objects, and more complex logical expressions.

Key features of OCL include:

Expressiveness: OCL allows for the expression of a wide range of constraints and conditions using a formal syntax.

Formality: OCL is a formal language, meaning that its syntax and semantics are precisely defined, allowing for rigorous analysis and verification of models.

Integration with Modeling Languages: OCL is designed to be integrated with modeling languages such as UML and MOF, allowing constraints to be specified directly within models.

Tool Support: There are various tools and environments that support the use of OCL for specifying and analyzing models, including model editors and validation tools.

Compatibility: OCL is designed to work with a variety of modeling languages and platforms, making it a versatile choice for specifying constraints in different contexts.

Designing Business Classes:



Designing business classes involves creating a set of classes that represent the various entities and concepts within a business domain. These classes encapsulate both data and behaviour relevant to the business processes and operations. Here are some steps and considerations for designing business classes:

Identify Business Entities: Begin by identifying the key entities or objects within the business domain. These could be tangible entities like customers, products, orders, or intangible concepts like invoices, transactions, or accounts.

Define Attributes: For each identified entity, define the attributes or properties that describe it. Attributes represent the data associated with an object. For example, a "Customer" class may have attributes such as name, address, email, etc.

Identify Behaviors: Determine the behaviors or operations that the classes will need to perform. These behaviors can include actions such as creating, updating, deleting objects, as well as specific business-related operations like processing orders or generating invoices.

Encapsulation: Encapsulate the attributes and behaviors within the appropriate classes. Encapsulation ensures that data and behavior are bundled together within a class, providing better control over access and manipulation.

Define Relationships: Identify and define the relationships between different classes. Relationships represent the associations or connections between entities. For example, a "Product" class may have a relationship with an "Order" class indicating that a product is part of an order.

Inheritance and Polymorphism: Consider if there are common attributes or behaviors that can be abstracted into a superclass or interface. This promotes reuse and maintains consistency across related classes. Additionally, utilize polymorphism to allow different classes to respond to the same message or method in different ways.

Validation and Constraints: Implement validation logic within the classes to ensure that the data remains consistent and valid. This can include checks for data integrity, business rules, and constraints.

Designing Well Defined Class Visibility:



In object-oriented programming, class visibility refers to the accessibility of class members (attributes and methods) from other classes or modules. Designing well-defined class visibility is crucial for creating maintainable, secure, and easily understandable code. Here are some guidelines for designing class visibility:

Encapsulation: Encapsulation is a fundamental principle of object-oriented design that promotes hiding the internal state of an object and only allowing access through well-defined interfaces (methods). Class members should be encapsulated by default, meaning they should be private unless there is a specific reason for them to be accessible from outside the class.

Use Access Modifiers: Most object-oriented programming languages provide access modifiers like public, private, protected, and sometimes internal or package-private. Use these access modifiers to explicitly specify the visibility of class members.

public: Accessible from any other class or module.

private: Accessible only within the declaring class.

protected: Accessible within the declaring class and its subclasses.

internal (if available): Accessible within the same module.

package-private (if available): Accessible within the same package.

Minimize Public Interface: Minimize the number of public members exposed by a class. This reduces the surface area of the class's interface and makes it easier to understand and maintain. Only expose members that are necessary for external interaction.

Favor Composition over Inheritance: Favor composition over inheritance when designing class relationships. This means that instead of exposing internal details through inheritance, prefer encapsulating related functionality within separate classes and providing access to them through well-defined interfaces.

Information Hiding: Hide implementation details that are not relevant to the class's external users. Expose only the necessary abstractions and hide internal implementation details that may change over time.



Limit Mutability: Limit the mutability of class members, especially those that are publicly accessible. Minimize the exposure of mutable state and prefer immutability where possible to reduce the risk of unintended side effects.

Follow the Principle of Least Privilege: Follow the principle of least privilege, which states that a class should have only the minimum level of access required to perform its intended functionality. Restrict access to sensitive or implementation-specific details.

Documentation: Clearly document the intended usage of public class members, including their purpose, expected behavior, and any preconditions or postconditions. This helps other developers understand how to interact with the class correctly.

Code Reviews and Refactoring: Conduct regular code reviews to ensure that class visibility is well-defined and adheres to design principles. Refactor code as needed to improve class visibility and maintainability.

Consideration for Persistence: If the business classes need to interact with a database or persist data in any way, consider how the classes will be mapped to database tables or other persistence mechanisms.

Testability and Modifiability: Design classes in a way that promotes testability and modifiability. Use techniques such as dependency injection and separation of concerns to isolate dependencies and make classes easier to test and maintain.

Iterative Refinement: Designing business classes is often an iterative process. Refine and iterate on the design based on feedback, changing requirements, and evolving business needs.

Attribute Refinement:

Attribute refinement is a process in software design where the attributes of a class or entity are defined and detailed to better represent the characteristics or properties of that class or entity. This process involves breaking down high-level attributes into more specific and refined attributes that accurately capture the requirements and nuances of the system being modeled.

Here are some steps involved in attribute refinement:



Identify High-Level Attributes: Begin by identifying the high-level attributes of the class or entity. These attributes represent the general characteristics or properties that need to be captured.

Analyze Requirements: Analyze the requirements of the system or domain to understand the specific details and variations that need to be captured by the attributes.

Decompose Attributes: Break down the high-level attributes into more specific and refined attributes that capture the detailed aspects of the class or entity. Consider different categories or aspects of the attribute that need to be represented.

Define Data Types: Define appropriate data types for each refined attribute based on the nature of the data it represents. Consider using primitive data types, custom data types, or data structures as needed.

Specify Constraints: Specify any constraints or rules that apply to the refined attributes, such as range limits, format requirements, or validation rules.

Consider Relationships: Consider the relationships between attributes within the class or entity. Determine if there are dependencies or interactions between attributes that need to be represented.

Document Attribute Definitions: Document the definitions and details of each refined attribute to ensure clarity and understanding among developers and stakeholders.

Validate Attribute Refinement: Validate the refined attributes against the requirements and constraints of the system to ensure that they accurately capture the necessary details and variations.

Iterative Process: Attribute refinement is often an iterative process that involves refining and adjusting attributes based on feedback, changing requirements, and evolving understanding of the system.

Method Design Using UML Activity Diagram:

Designing methods using UML activity diagrams involves visually representing the flow of control and data between various activities or actions within a



method. Activity diagrams in UML (Unified Modeling Language) are graphical representations that depict the flow of control from one activity to another within a system or process. Here's how you can design methods using UML activity diagrams:

Identify Activities: Begin by identifying the activities or steps that make up the method. Each activity represents a specific action or task within the method.

Define Activity Nodes: In the activity diagram, represent each activity as an activity node. Activity nodes can be depicted as rounded rectangles or other shapes, with labels indicating the name of the activity.

Connect Activities with Control Flows: Use control flows (arrows) to connect the activity nodes and depict the flow of control between them. Control flows indicate the sequence in which activities are performed within the method.

Include Decision Points: If the method involves making decisions based on certain conditions, include decision points in the activity diagram. Decision points are depicted as diamonds, with outgoing control flows representing the possible paths based on the decision outcome.

Include Merge Nodes: Merge nodes are used to synchronize multiple incoming control flows into a single outgoing flow. Include merge nodes where necessary to combine control flows that converge at certain points in the method.

Specify Actions and Behaviors: Optionally, you can specify actions and behaviors associated with each activity node. These can be depicted using action symbols within the activity node, indicating the specific behavior or action to be performed.

Include Swimlanes: If the method involves multiple actors or entities, you can use swimlanes to partition the activities based on the responsible entity. Swimlanes visually separate the activities performed by different actors within the method.

Specify Data Flow: Optionally, you can specify data flow between activities using data flow arrows. Data flow arrows indicate the movement of data between different activities or steps within the method.



Annotate the Diagram: Use annotations and comments to provide additional information about the method, such as preconditions, post conditions, or any other relevant details.

Validate and Review: Review the activity diagram to ensure that it accurately represents the flow of control and data within the method. Validate the diagram against the requirements and specifications of the method.

Packaging and Managing Classes:

Packaging and managing classes is an important aspect of software development, especially in object-oriented programming languages where classes are the building blocks of the system. Here are some guidelines for packaging and managing classes effectively:

Logical Organization: Group related classes together in packages based on their functionality or domain. This logical organization makes it easier for developers to find and understand classes related to specific features or components of the system.

Avoid Overly Large Packages: Keep packages small and focused to prevent them from becoming too large and unwieldy. Large packages can be difficult to manage and understand, leading to decreased maintainability.

Encapsulation: Encapsulate related classes within packages to hide their implementation details and promote modularity. Only expose the necessary interfaces and public classes to other packages.

Dependency Management: Manage dependencies between packages carefully to minimize coupling and ensure high cohesion. Minimize dependencies between packages to promote reusability and maintainability.

Use of Access Modifiers: Use access modifiers (e.g., public, private, protected) effectively to control the visibility of classes and members within packages. Encapsulate implementation details by making classes and members private or package-private whenever possible.

Package Naming Conventions: Follow consistent and meaningful naming conventions for packages to make it easier to understand their purpose and contents. Use lowercase letters and meaningful names that reflect the functionality or domain of the classes within the package.



Package Documentation: Document packages and their contents using comments and documentation tools to provide information about their purpose, usage, and dependencies. This helps developers understand how to use the classes within the package correctly.

Versioning and Release Management: Manage versions and releases of packages carefully to ensure backward compatibility and facilitate software updates. Use version control systems and package management tools to track changes and manage dependencies between packages.

Testing and Quality Assurance: Test packages independently to ensure their correctness and functionality. Use unit tests, integration tests, and other testing techniques to verify the behavior of classes within packages.

Refactoring and Maintenance: Regularly review and refactor packages to improve their structure, readability, and maintainability. Remove unnecessary dependencies, split large packages, and consolidate related functionality to keep the codebase clean and manageable.

Designing Access Layer:

Object Relational Systems:

Object-Relational Systems (also known as Object-Relational Mapping, ORM) are software frameworks or libraries that facilitate the mapping between an object-oriented programming language and a relational database management system (RDBMS). These systems bridge the gap between the object-oriented model used in application code and the relational model used in databases, allowing developers to work with objects in their code while persisting data in a relational database.

Here are some key aspects and features of Object-Relational Systems:

Mapping Objects to Tables: Object-Relational Systems provide mechanisms to map objects from the application domain to tables in a relational database. This mapping typically involves defining how object attributes and relationships correspond to database columns and tables.

Support for Inheritance and Polymorphism: Many Object-Relational Systems support mapping inheritance hierarchies and polymorphic associations between



objects to corresponding structures in the database. This allows developers to model complex object-oriented relationships in their applications.

Querying and Retrieving Data: Object-Relational Systems often provide query languages or query APIs that allow developers to retrieve data from the database using object-oriented syntax. These query mechanisms typically abstract the underlying SQL language and provide a more intuitive way to work with data.

Transaction Management: Object-Relational Systems often provide facilities for managing database transactions within the context of object-oriented code. This includes support for transaction demarcation, isolation levels, and error handling.

Caching and Performance Optimization: Many Object-Relational Systems include features for caching objects in memory to improve performance and reduce the number of database round-trips. This can include strategies such as first-level and second-level caching.

Schema Evolution and Migration: Object-Relational Systems often include tools and utilities for managing database schema evolution and migration. This allows developers to evolve the database schema over time while maintaining compatibility with existing application code.

Concurrency Control: Object-Relational Systems typically provide mechanisms for managing concurrency and data consistency in multi-user environments. This includes support for optimistic locking, pessimistic locking, and versioning.

Integration with Frameworks and Tools: Many Object-Relational Systems integrate with popular application development frameworks and tools, making it easier to use them in the context of modern software development practices.

Examples of Object-Relational Systems include Hibernate for Java, Entity Framework for .NET, SQL Alchemy for Python, and Active Record for Ruby on Rails. These systems abstract the complexities of working with relational databases in object-oriented applications, allowing developers to focus on application logic and domain modelling.

Object Relation Mapping:



Object-Relational Mapping (ORM) is a programming technique that facilitates the conversion of data between incompatible type systems, particularly between object-oriented programming languages and relational databases. The primary goal of ORM is to bridge the gap between the object-oriented model used in application code and the relational model used in databases, allowing developers to work with objects in their code while transparently persisting and retrieving data from a relational database.

Here's how ORM typically works:

Mapping Entities to Tables: In ORM, each class or entity in the object-oriented programming language corresponds to a table in the relational database. Attributes of the class are mapped to columns in the table, and relationships between classes are represented using foreign key constraints.

Querying with Object-Oriented Syntax: ORM frameworks provide APIs or query languages that allow developers to query and manipulate data using object-oriented syntax. Developers can perform CRUD (Create, Read, Update, Delete) operations on objects using familiar object-oriented concepts, abstracting away the need to write SQL queries directly.

Relationship Mapping: ORM frameworks support mapping relationships between objects, such as one-to-one, one-to-many, and many-to-many relationships, to corresponding relationships in the database schema. This includes mapping associations between classes using foreign keys and join tables.

Transaction Management: ORM frameworks typically provide facilities for managing transactions, allowing developers to perform multiple database operations within a single transaction and ensuring data consistency and integrity.

Identity Mapping: Identity mapping involves mapping the identity of objects in the application domain to primary keys in the database tables. This ensures that each object instance is uniquely identified and can be persisted and retrieved correctly.

Lazy Loading and Eager Loading: ORM frameworks often support lazy loading and eager loading mechanisms to efficiently retrieve related objects from the database, reducing the number of database queries and improving performance.



Optimistic and Pessimistic Locking: ORM frameworks support locking mechanisms for managing concurrency and data consistency in multi-user environments, including optimistic and pessimistic locking strategies.

Integration with Object-Oriented Programming Languages: ORM frameworks are typically designed to seamlessly integrate with popular object-oriented programming languages like Java, C#, Python, and Ruby, providing a natural and consistent development experience.

ORM frameworks abstract the complexities of database interactions, allowing developers to focus on application logic and domain modeling in their object-oriented code. Examples of ORM frameworks include Hibernate for Java, Entity Framework for .NET, SQLAlchemy for Python, and ActiveRecord for Ruby on Rails. These frameworks provide powerful tools for working with relational databases in object-oriented applications.

Table Class Mapping:

Table class mapping, also known as entity mapping, is a fundamental concept in Object-Relational Mapping (ORM) systems. It involves defining the relationship between database tables and object-oriented classes in a software application. The goal is to facilitate seamless interaction between the application's object-oriented model and the relational database, enabling developers to work with objects in their code while transparently persisting and retrieving data from the database.

Here are the key components and considerations of table class mapping:

Entity Classes: In object-oriented programming, entity classes represent the various entities or objects in the application domain. These classes encapsulate both data (attributes or properties) and behavior (methods or functions) related to the entities they represent.

Database Tables: In relational databases, tables are used to store data in a structured format. Each table consists of columns (fields) and rows (records), where columns represent attributes and rows represent individual data instances.

Mapping Attributes to Columns: One of the primary tasks in table class mapping is mapping the attributes of an entity class to the columns of a



corresponding database table. Each attribute in the class typically corresponds to a column in the table.

Mapping Relationships: In addition to mapping attributes, table class mapping also involves mapping relationships between entities/classes. For example, if an entity has a one-to-many or many-to-many relationship with another entity, this relationship needs to be mapped to corresponding foreign key constraints in the database schema.

Data Types Mapping: Object-oriented programming languages often support a wide range of data types, while relational databases have their own set of data types. Mapping between these data types requires careful consideration to ensure compatibility and data integrity.

Identity Mapping: Identity mapping involves mapping the identity of objects in the application domain to primary keys in the database tables. This ensures that each object instance is uniquely identified and can be persisted and retrieved correctly.

Mapping Annotations or Configuration: ORM frameworks provide mechanisms for specifying the mapping between entity classes and database tables. This can be done using annotations (in languages like Java and C#) or configuration files (in languages like XML or YAML).

Bidirectional Mapping: In some cases, it's necessary to establish a bidirectional mapping between entity classes and database tables, allowing changes made to the object model to be reflected in the database schema, and vice versa.

Table — Inherited Classes Mapping:

Mapping inherited classes to database tables is a fundamental aspect of Object-Relational Mapping (ORM) systems, especially when dealing with inheritance hierarchies in object-oriented programming languages. Inheritance mapping allows developers to model complex class hierarchies in their object-oriented code while persisting and retrieving data from a relational database.

Here's how table mapping for inherited classes is typically handled in ORM systems:

Single Table Inheritance (STI):



In STI, all classes in the inheritance hierarchy are mapped to a single database table.

A discriminator column is used to indicate the specific subclass type for each row in the table.

All attributes of the superclass and its subclasses are stored in the same table, with NULL values for attributes that are not applicable to certain subclasses.

Class Table Inheritance (CTI):

In CTI, each class in the inheritance hierarchy is mapped to its own database table.

Each table represents a specific subclass, and it includes only the attributes that are specific to that subclass.

The superclass is also mapped to its own table, containing only the attributes that are common to all subclasses.

Relationships between subclasses and the superclass are typically modeled using foreign key constraints.

Concrete Table Inheritance (CTI):

In CTI, each concrete class (i.e., non-abstract class) in the inheritance hierarchy is mapped to its own database table.

Each table contains all attributes of the corresponding class, including both superclass and subclass attributes.

There is no superclass table; instead, each concrete subclass has its own table with duplicated superclass attributes.

Joined Table Inheritance (JTI):

In JTI, each class in the inheritance hierarchy is mapped to its own database table, similar to CTI.

Additionally, there is a separate table for the superclass, containing only the attributes that are common to all subclasses.



Each subclass table includes a foreign key constraint referencing the superclass table, establishing a one-to-one relationship between each subclass table and the superclass table.

ORM frameworks provide mechanisms for specifying inheritance mapping strategies and handling the mapping details automatically. Developers can use annotations, configuration files, or fluent API methods to define the inheritance mapping strategy and customize the mapping behavior as needed.

Designing the Access Layer Classes:

Create mirror classes:

Creating mirror classes typically involves generating or writing classes that have the same structure and attributes as existing classes in a software system. Mirror classes serve various purposes, such as:

Data Transformation: Mirror classes are sometimes used to transform data between different systems or formats. For example, when integrating with external systems that have different data representations, mirror classes can be created to map data between the two systems.

Serialization and Deserialization: In some cases, mirror classes are used for serialization and deserialization purposes. For instance, when serializing objects to JSON or XML formats, mirror classes may be employed to define the structure of the serialized data.

Testing: Mirror classes can also be used in testing scenarios, particularly when testing against mock or stub objects. By creating mirror classes that mimic the behavior of real classes, developers can isolate components for testing purposes.

Legacy Code Integration: When working with legacy codebases or systems that lack proper abstraction, mirror classes can be created to encapsulate the behavior of existing classes while providing a cleaner interface for new development.

Dynamic Code Generation: Mirror classes can be dynamically generated at runtime based on certain criteria or configurations. This is often used in frameworks or libraries that rely on runtime code generation.



Code Generation Tools: Some code generation tools automatically generate mirror classes based on predefined templates or schemas. These tools are commonly used in environments where boilerplate code needs to be minimized.

The process of creating mirror classes varies depending on the specific use case and requirements. In some cases, mirror classes may be manually created by developers, while in other cases, they may be generated automatically using code generation tools or frameworks.

Here's a simple example of manually creating mirror classes in Java:

```
public class Person {  
    private String name;  
    private int age;  
    // Getters and setters  
}  
  
public class PersonMirror {  
    private String name;  
    private int age;  
    // Getters and setters  
}
```

In this example, Person Mirror is a mirror class of Person, having the same structure and attributes. Depending on the specific use case, additional logic or transformations may be added to mirror classes as needed.

Identify Access layer class relationships:

In software engineering, the access layer (also known as the data access layer or persistence layer) is responsible for managing the interaction between the application and the underlying data storage, such as a database. Class relationships within the access layer typically involve classes that represent data entities, data access objects (DAOs), and other components responsible for data access and manipulation.



Here are some common class relationships within the access layer:

Entity Classes: Entity classes represent the data entities or domain objects that are stored in the database. These classes typically map directly to database tables and contain attributes that correspond to table columns. Class relationships within the access layer may include:

One-to-One relationships between different entity classes representing related data entities.

One-to-Many relationships between an entity class and other related entities, where each instance of the entity class can be associated with multiple instances of the related entity class.

Many-to-Many relationships between two entity classes, where instances of each class can be associated with multiple instances of the other class.

Data Access Objects (DAOs): DAOs are responsible for abstracting the interaction with the underlying data storage, such as querying and updating data in the database. Class relationships within the access layer may include:

Association relationships between DAO classes and entity classes, where DAOs are responsible for retrieving, creating, updating, and deleting instances of entity classes.

Dependency relationships between DAO classes and other components such as data source managers or connection pools, which provide the necessary infrastructure for database connectivity.

Data Transfer Objects (DTOs): DTOs are used to transfer data between different layers of the application, often between the access layer and the business logic layer. Class relationships within the access layer may include:

Association relationships between DTO classes and entity classes, where DTOs are used to transfer data between the access layer and other layers of the application.

Dependency relationships between DTO classes and data access components such as DAOs, which use DTOs to transfer data between the application and the database.



Helper Classes and Utilities: In addition to entity classes, DAOs, and DTOs, the access layer may include helper classes and utilities responsible for tasks such as database connection management, query execution, and data transformation. Class relationships within the access layer may include:

Association relationships between helper classes and DAOs or entity classes, where helper classes provide support functions for data access operations.

Dependency relationships between utilities and other components within the access layer, where utilities are used to perform specific data access-related tasks.

These are some common class relationships within the access layer of a software application. Depending on the specific architecture and design of the application, additional class relationships and patterns may be employed to manage data access effectively.

Eliminate redundant classes:

Eliminating redundant classes in a software system is essential to maintain a clean, efficient, and maintainable codebase. Redundant classes are classes that duplicate functionality or serve no purpose in the system. Here are some steps you can take to identify and eliminate redundant classes:

Review Class Functionality: Review the functionality of each class in the system to identify any duplication or overlap. Look for classes that provide similar or identical functionality.

Identify Common Functionality: Identify common functionality that is duplicated across multiple classes. This could include methods, attributes, or behavior that can be abstracted into a single class or shared utility.

Use Inheritance and Composition: Use inheritance and composition to eliminate redundant classes. If multiple classes share common functionality, consider creating a superclass that contains the shared behavior. Alternatively, you can extract common functionality into a separate class and use composition to include it in other classes.

Refactor Code: Refactor the codebase to remove redundant classes and consolidate functionality where possible. This may involve merging classes,



extracting common functionality into shared components, or redesigning class hierarchies.

Consolidate Similar Classes: If multiple classes provide similar functionality but with slight variations, consider consolidating them into a single class with configurable options or parameters to handle the variations.

Use Design Patterns: Utilize design patterns such as Singleton, Factory, or Strategy patterns to eliminate redundancy and improve code reuse where applicable.

Review Dependencies: Review the dependencies between classes to identify any unnecessary or redundant dependencies. Remove dependencies on classes that are no longer needed or replace them with more appropriate dependencies.

Unit Testing: Use unit tests to ensure that refactoring and eliminating redundant classes do not introduce regressions or break existing functionality. Test the modified code to verify that it behaves as expected after the changes.

Code Reviews: Conduct code reviews with team members to review changes and discuss potential areas for eliminating redundancy. Peer review can help identify redundant classes and suggest alternative approaches for consolidation.

Document Changes: Document any changes made to the codebase to eliminate redundant classes, including the reasons for the changes and the impact on the overall system architecture.

By following these steps and systematically identifying and eliminating redundant classes, you can improve the maintainability, readability, and efficiency of your software system.

Create method classes:

Creating method classes involves designing classes that encapsulate specific behaviors or operations within a software system. These classes typically contain methods (or functions) that perform specific tasks related to a particular functionality. Here's how you can create method classes:

Identify Functionalities: Start by identifying the different functionalities or operations that need to be implemented within your software system. Each functionality may require one or more method classes to encapsulate its behavior.



Define Responsibilities: Define the responsibilities of each method class. What specific tasks or operations will the class be responsible for? This will help in determining the scope and functionality of the class.

Encapsulation: Encapsulate related behaviors or operations within each method class. This means grouping together methods that are closely related to perform a specific task or achieve a particular goal.

Class Structure: Design the structure of each method class. Define the attributes (if any) and methods that the class will contain. Consider the inputs, outputs, and dependencies of each method when designing the class structure.

Method Implementation: Implement the methods within each method class to perform the desired functionality. Each method should have a clear purpose and should follow the single responsibility principle (i.e., each method should do one thing and do it well).

Error Handling: Implement error handling within the methods to handle unexpected situations or errors gracefully. This may include checking for invalid inputs, handling exceptions, and returning meaningful error messages.

Testability: Design method classes with testability in mind. Ensure that methods are modular and can be easily tested in isolation using unit tests. Consider using dependency injection to facilitate testing and mocking dependencies.

Documentation: Document the purpose, inputs, outputs, and usage of each method within the method class. This documentation will help other developers understand how to use the class and its methods correctly.

Naming Conventions: Follow naming conventions for methods and classes to make the codebase more readable and maintainable. Use descriptive names that accurately reflect the purpose of each method class and its methods.

Refactoring and Iteration: Refactor the method classes as needed based on feedback, changing requirements, and evolving understanding of the system. Iteratively improve the design and implementation of the method classes to meet the needs of the software system.

Designing View Layer:

When designing the view layer of a software application, especially in object-oriented programming, you'll typically organize your code into classes that

Approved by AICTE, New Delhi vide letter No. F- 27-29 / 91 / AICTE / US (PG) dt. 20/09/93

Address : A/p. Loni Bk., Tal. Rahata, Dist. Ahmednagar (M.S.) PIN: 413736

Ph No.: (O) +91-2422-273539 / 273203 / (P) 273204 / 273463

Website: www.pravaraengg.org.in , www.pravara.in | Email: principal.precloni@pravara.in



represent different components of the user interface. These classes interact with each other to present information to users and capture their input. Here's a basic outline of how you might design classes within the view layer:

View Classes:

MainWindow: This class represents the main window of the application and may contain other view components, such as menus, toolbars, and status bars.

Dialogs: Classes representing various dialog boxes or modal windows used for user input or displaying messages.

Panels: Classes representing panels or sections within the main window, used for organizing and displaying different parts of the user interface.

Widget Classes:

Button: Represents a clickable button that triggers actions when clicked.

Label: Displays static text or images to the user.

Text Field / Text Area: Allows users to input and edit text.

Checkbox / Radio Button: Represents options that users can select or deselect.

Combo Box / List Box: Displays a list of options for users to choose from.

Controller Classes:

Event Handlers: Classes responsible for handling user interactions, such as button clicks, mouse events, and keyboard input.

Listeners / Observers: Classes that listen for changes in the view components and respond accordingly, such as updating the display or triggering actions.

Model-View-Controller (MVC) Components:

Model Classes: Represents the data and business logic of the application. These classes are not directly part of the view layer but are accessed by view components to retrieve or manipulate data.

Controller Classes: Handle user input and interact with model classes to update data. They may also trigger updates to the view components.



View Classes: Represented by the UI components mentioned earlier, responsible for displaying data to users and capturing their input. They may also communicate with controllers to handle user actions.

Utility Classes:

Formatter: Helps format data for display in the UI, such as formatting dates, numbers, or currency.

Validator: Validates user input to ensure it meets certain criteria or constraints.

Custom Components:

Custom Widgets: Classes representing custom-designed UI components tailored to specific application requirements.

Composite Components: Classes that combine multiple basic UI components to create more complex, custom widgets.

When designing these classes, consider factors such as encapsulation, modularity, reusability, and separation of concerns to ensure a well-structured and maintainable codebase. Additionally, adhere to design principles such as the Single Responsibility Principle (SRP) and the Open/Closed Principle (OCP) to keep your classes focused and adaptable to change.

Identifying View Classes by Analysing Use Cases:

To identify view classes by analysing use cases, you'll need to understand the requirements of your application and how users will interact with it. Use cases describe the interactions between users (actors) and the system to achieve specific goals or tasks. By analysing these use cases, you can identify the view classes needed to support the user interface. Here's how you can do it:

Identify Actors: Begin by identifying the different actors (users or external systems) involved in your application. Each actor represents a role that interacts with the system to accomplish specific tasks.

List Use Cases: Identify and list all the use cases that describe the interactions between actors and the system. A use case describes a specific goal or task that a user wants to accomplish using the system.



Analyze User Interactions: For each use case, analyze the user interactions required to complete the task. Consider the inputs required from the user, the actions they need to perform, and the feedback or results they expect from the system.

Identify UI Components: Based on the user interactions identified in step 3, identify the UI components (e.g., buttons, text fields, labels) needed to support those interactions. Each UI component represents a potential view class.

Group Related UI Components: Group related UI components together based on the use cases they support. For example, UI components related to logging in might be grouped together in a login view class.

Define View Classes: Based on the grouped UI components, define the view classes needed to implement the user interface. Each view class should encapsulate the UI components and related logic required to support a specific use case or set of use cases.

Refine and Iterate: Refine your analysis based on feedback from stakeholders and usability testing. Iterate on the design to ensure that the view classes effectively support the user interactions described in the use cases.

Consider Reusability: Identify opportunities for reusing view classes across multiple use cases or scenarios. Design view classes to be modular and reusable to minimize duplication and improve maintainability.

Align with UI Design Guidelines: Ensure that the design of view classes aligns with UI design guidelines and best practices to create a user-friendly and consistent user interface.

Document Class Responsibilities: Document the responsibilities of each view class, including the use cases it supports and the interactions it facilitates. This documentation will help guide the implementation and future maintenance of the view layer.

Macro-Level Design Process – identify view layer objects:

The macro-level design process involves identifying and defining the high-level components and structures of a software system. When identifying view layer objects as part of this process, you're essentially determining the key



components and classes responsible for presenting information to users and capturing their input. Here's how you can approach it:

Understand Requirements: Begin by thoroughly understanding the requirements of the system, including the functional and non-functional requirements related to the user interface. This involves gathering input from stakeholders, analyzing user stories, and reviewing any existing documentation.

Identify User Roles and Tasks: Identify the different user roles or personas that will interact with the system and the tasks they need to accomplish. This helps in understanding the various user interactions and workflows that the view layer needs to support.

Define Use Cases: Define the various use cases or scenarios that describe how users will interact with the system to achieve their goals. Use cases provide a structured way to understand the functional requirements of the system from a user's perspective.

Identify View Layer Components: Based on the use cases and user interactions identified, determine the key components and objects needed in the view layer to support the user interface. This may include:

Screens or Pages: Identify the different screens or pages that users will interact with. Each screen represents a distinct view in the application.

Widgets or Controls: Identify the various UI elements or controls (e.g., buttons, text fields, dropdowns) that users will interact with on each screen.

Layouts: Define the layout or arrangement of UI elements within each screen or page. Consider factors such as navigation, grouping of related elements, and overall visual hierarchy.

Data Binding: Determine how data will be displayed and manipulated within the user interface. Identify any data-bound elements that need to be synchronized with underlying data models.

Define Relationships and Interactions: Define the relationships and interactions between view layer components. Determine how components communicate with each other and how user actions trigger changes in the interface.



Consider UI Patterns and Guidelines: Consider established UI design patterns, principles, and guidelines to inform the design of view layer objects. This includes principles of usability, consistency, accessibility, and responsive design.

Modularize and Reuse: Design view layer objects to be modular and reusable where possible. Identify common UI elements or patterns that can be abstracted into reusable components to promote consistency and reduce duplication.

Document Design Decisions: Document the design decisions related to view layer objects, including their responsibilities, relationships, and interactions. This documentation serves as a reference for development and future maintenance.

Review and Iterate: Review the design of view layer objects with stakeholders, UX designers, and development team members to gather feedback and iterate on the design as needed. Consider usability testing and prototyping to validate the design with end users.

Build prototype for view layer Interface:

Building a prototype for the view layer interface involves creating a visual representation of the user interface (UI) components and interactions based on the design specifications and requirements. Prototyping helps validate the design concepts, gather feedback from stakeholders, and identify potential improvements before proceeding with full-scale development. Here's how you can build a prototype for the view layer interface:

Select a Prototyping Tool: Choose a prototyping tool that best suits your needs and preferences. There are various prototyping tools available, ranging from simple wire framing tools to more advanced interactive prototyping platforms. Some popular options include Adobe XD, Sketch, Figma, InVision, Axure RP, and Balsamiq.

Create Wireframes: Start by creating wireframes or low-fidelity mockups of the UI components using the selected prototyping tool. Wireframes are simplified representations of the UI layout, structure, and content, focusing on the placement of elements rather than detailed visual design. Use basic shapes, text placeholders, and simple icons to outline the main components of each screen or page.



Define Navigation Flow: Define the navigation flow between different screens or pages within the prototype. Identify the sequence of interactions and transitions that users will follow when navigating through the application. Use arrows or links to indicate the flow between screens and simulate user interactions.

Add Interactive Elements: Enhance the wireframes by adding interactive elements to simulate user interactions and functionality. Incorporate clickable buttons, links, menus, tabs, and other interactive components to demonstrate how users can interact with the interface. Use prototyping tools' interactive features to define the behavior of these elements, such as linking buttons to navigate between screens or triggering pop-up dialogs.

Incorporate Visual Design: Once the wireframes and interactive elements are in place, incorporate visual design elements to enhance the prototype's appearance and aesthetics. Add colors, typography, icons, images, and other visual elements to create a more polished and visually appealing UI. Ensure consistency with the brand's visual identity and adhere to UI design principles and guidelines.

Test and Iterate: Test the prototype with stakeholders, end users, or usability testers to gather feedback and identify areas for improvement. Use the feedback to iterate on the design, making necessary adjustments to enhance usability, clarity, and user experience. Continue refining the prototype through multiple iterations until it accurately represents the desired UI design and functionality.

Document Design Decisions: Document the design decisions, feedback received, and iterations made during the prototyping process. This documentation serves as a reference for development and future iterations of the UI design.

Present and Validate: Present the prototype to stakeholders, including product owners, developers, designers, and other project stakeholders. Validate the design concepts, gather buy-in from stakeholders, and obtain approval to proceed with development based on the prototype.

Test Usability and User satisfaction:

Component and Deployment Diagrams are part of the Unified Modeling Language (UML) and are used to depict the architecture of a software system,



including its components and their deployment on hardware nodes. Here's how you can design component and deployment diagrams:

Identify Components: Begin by identifying the major components of your system. Components represent the modular units of functionality within the system, often corresponding to classes, modules, or packages in the codebase. Examples of components include user interfaces, business logic modules, data access layers, external services, etc.

Define Component Relationships: Determine how the components interact with each other. Identify dependencies and relationships between components, such as dependencies for functionality, communication protocols, or data flow.

Create Component Diagram: Use a component diagram to visually represent the components of the system and their relationships. In a component diagram:

Draw boxes to represent each component, labeling them with their names.

Use connectors (lines) to represent relationships between components, such as dependencies or associations.

Add interfaces to specify the points of interaction between components.

Identify Deployment Nodes: Determine the hardware nodes on which your system will be deployed. Nodes represent physical or virtual computing resources, such as servers, databases, client devices, etc.

Define Deployment Relationships: Determine how the components will be deployed on the hardware nodes. Identify deployment dependencies and relationships, such as which components run on which nodes, communication paths between nodes, etc.

Create Deployment Diagram: Use a deployment diagram to visually represent the deployment architecture of the system. In a deployment diagram:

Draw nodes to represent the hardware nodes, labeling them with their names (e.g., server names, device names).

Draw components and associate them with the nodes on which they will be deployed.

Use connectors (lines) to represent communication paths between nodes, such as network connections or deployment dependencies.



Optionally, specify deployment artifacts (e.g., executable files, libraries) associated with components.

Analyze and Refine: Review the component and deployment diagrams to ensure that they accurately represent the system architecture. Analyze the diagrams for clarity, consistency, and completeness. Refine the diagrams as needed to address any inconsistencies or ambiguities.

Document: Document the design decisions, dependencies, and rationale behind the component and deployment architecture. This documentation serves as a reference for developers, architects, and other stakeholders involved in the project.

Review and Validate: Review the component and deployment diagrams with stakeholders, including developers, architects, and project managers. Validate the diagrams against the system requirements and make any necessary adjustments based on feedback.

By following these steps, you can effectively design component and deployment diagrams to depict the architecture of your software system, including its components and their deployment on hardware nodes. These diagrams serve as valuable tools for communication, documentation, and analysis throughout the software development lifecycle.